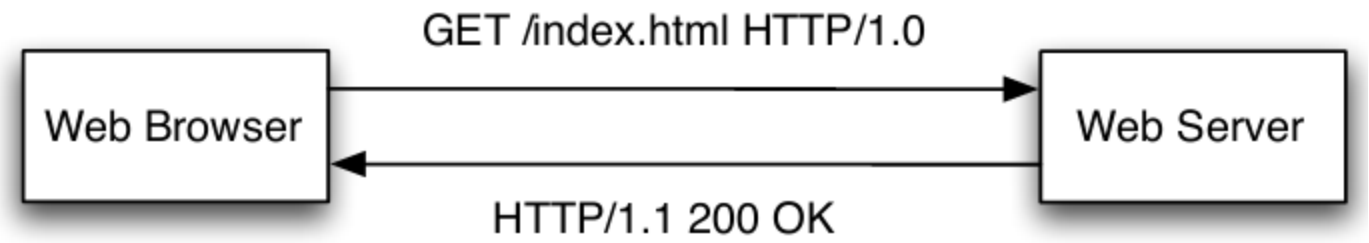
Web Technology

Lecture #1

World Wide Web

- The WWW is a service that operates over the internet.
- The World Wide Web (www) commonly known as the web, is a system of interlinked hypermedia documents accessed via the Internet.
- Hypertext Transfer Protocol (http) is the standard protocol for transferring web content
- HTTP specifies how clients request data, and how servers respond to these requests.

Client Server Model



- In this paradigm a client requests a server for a service and the server responds with the service.
- client–web browser
- ŝerver–web server

Client and Server

- **Client:** The software that resides on the remote machine, communicates with the server, fetches the information, processes it, and then displays it on the remote machine is called the client.
  - initiates contact with server (“speaks first”)
  - typically requests service from server
  - Web client implemented in browser

Client Server

- **Web Server:** Software that delivers Web pages and other documents to browsers using the HTTP protocol.
- **Web Page:** A web page is a document or resource of information that is suitable for the World Wide Web and can be accessed through a web browser.
- **URL:** Uniform Resource Locator, the unique address which identifies a resource on the Internet for routing purposes.

Client Server communication

1. The server listens on port 80 waiting for connections
2. The web browser connects to the server, and sends a request
3. The server responds with an error code or the web content

- **Activity on server side**

1. The server runs a program to generate the web content
2. This program gets run every time the given URL is requested
3. The server passes the http request details to the program
4. The program returns the web content or an error code

HTTP overview

- There are following three basic features which makes HTTP a simple but powerful protocol:
  - HTTP is connectionless
  - HTTP is media independent
  - HTTP is stateless
- The format of the request and response messages are similar, and consist of:
  - an initial line
  - zero or more header lines,
  - a blank line, and
  - an optional message body (e.g. a file, or query data, or query output).

Initial line

- Initial Request Line: A request line has three parts, separated by spaces: a *method* name, the path of the requested resource, and the version of HTTP being used. A typical request line is:

  GET /path/to/file/index.html HTTP/1.0

- The initial response line, called the status line, also has three parts separated by spaces: the HTTP version, a response status code that gives the result of the request, and an English reason phrase describing the status code. Typical status lines are:

  HTTP/1.0 200 OK

  or

  HTTP/1.0 404 Not Found

HTTP Request Methods

- The following are the most commonly used http method.
  1. The GET Method
  2. The POST Method
- **<u>GET Method:</u>**

http://www.test.com/index.htm?name1=value1&name2=value2

  - GET requests can be cached
  - GET requests remain in the browser history
  - GET requests can be bookmarked
  - GET requests should never be used when dealing with sensitive data
  - GET requests have length restrictions
  - GET requests should be used only to retrieve data

HTTP Request Methods

- **<u>POST Method</u>**
  - Information are embedded in the http header
  - POST requests are never cached
  - POST requests do not remain in the browser history
  - POST requests cannot be bookmarked
  - POST requests have no restrictions on data length

Status codes

Examples of status codes are:

- **100 Continue** Part of the request have been received
- **200 OK** The request succeeded, and the resulting resource (e.g. file or script output) is returned in the message body.
- **301 Moved Permanently** The requested page has moved to a new url .
- **404 Not Found** The requested resource doesn't exist.
- **500 Internal Server Error** An unexpected server error. The most common cause is a server-side script that has bad syntax, fails, or otherwise can't run correctly.

Header lines

- Header lines provide information about the request or response, or about the object sent in the message body.
- HTTP 1.0 defines 16 headers, though none are required. HTTP 1.1 defines 46 headers.
- Some of the commonly used header lines are:
  - The **From:** header gives the email address of whoever's making the request, or running the program doing so.
  - The **User-Agent:** header identifies the program that's making the request, in the form "**Program-name/x.xx**", where **x.xx** is the (mostly) alphanumeric version of the program. For example **User-agent: Mozilla/3.0Gold**.
  - The **Server:** header is analogous to the **User-Agent:** header: it identifies the server software in the form "**Program-name/x.xx**". For example, "**Server: Apache/1.2b3-dev**".
  - The **Last-Modified:** header gives the modification date of the resource that's being returned. It's used in caching and other bandwidth-saving activities. Use Greenwich Mean Time, in the format Last-Modified: Fri, 31 Dec 1999 23:59:59 GMT

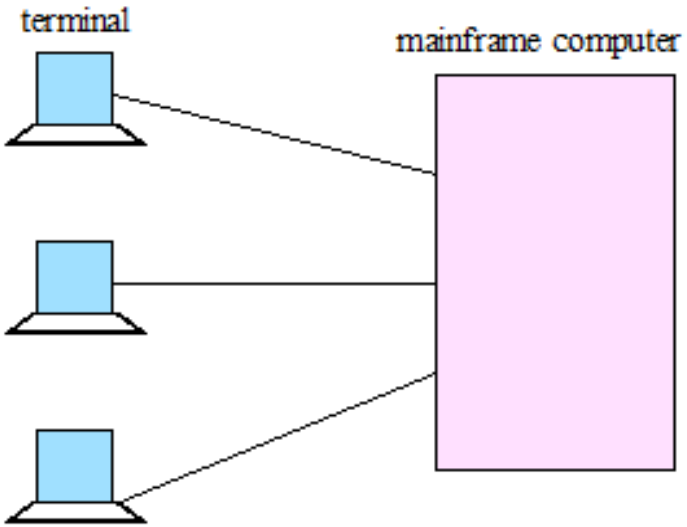
HTTP message body

- An HTTP message may have a body of data sent after the header lines. In a response, this is where the requested resource is returned to the client (the most common use of the message body), or perhaps explanatory text if there's an error. In a request, this is where user-entered data or uploaded files are sent to the server.
- If an HTTP message includes a body, there are usually header lines in the message that describe the body. In particular,
- The **Content-Type:** header gives the MIME-type of the data in the body, such as **text/html** or **image/gif**.
- The **Content-Length:** header gives the number of bytes in the body.

Sample HTTP Exchange

- To retrieve the file at the URL
- http://www.somehost.com/path/fie.html
- first open a socket to the host **www.somehost.com**. Then, send something like the following through the socket:
- GET /path/file.html HTTP/1.0
  From: someuser@jmarshall.com
  User-Agent: HTTPTool/1.0
  [blank line here]
- The server should respond with something like the following, sent back through the same socket:
- HTTP/1.0 200 OK
  Date: Fri, 31 Dec 1999 23:59:59 GMT
  Content-Type: text/html
  Content-Length: 1354
  <html> <body> <h1>Happy New Millennium!</h1> (more file contents) . . . </body> </html>

Centralized Processing



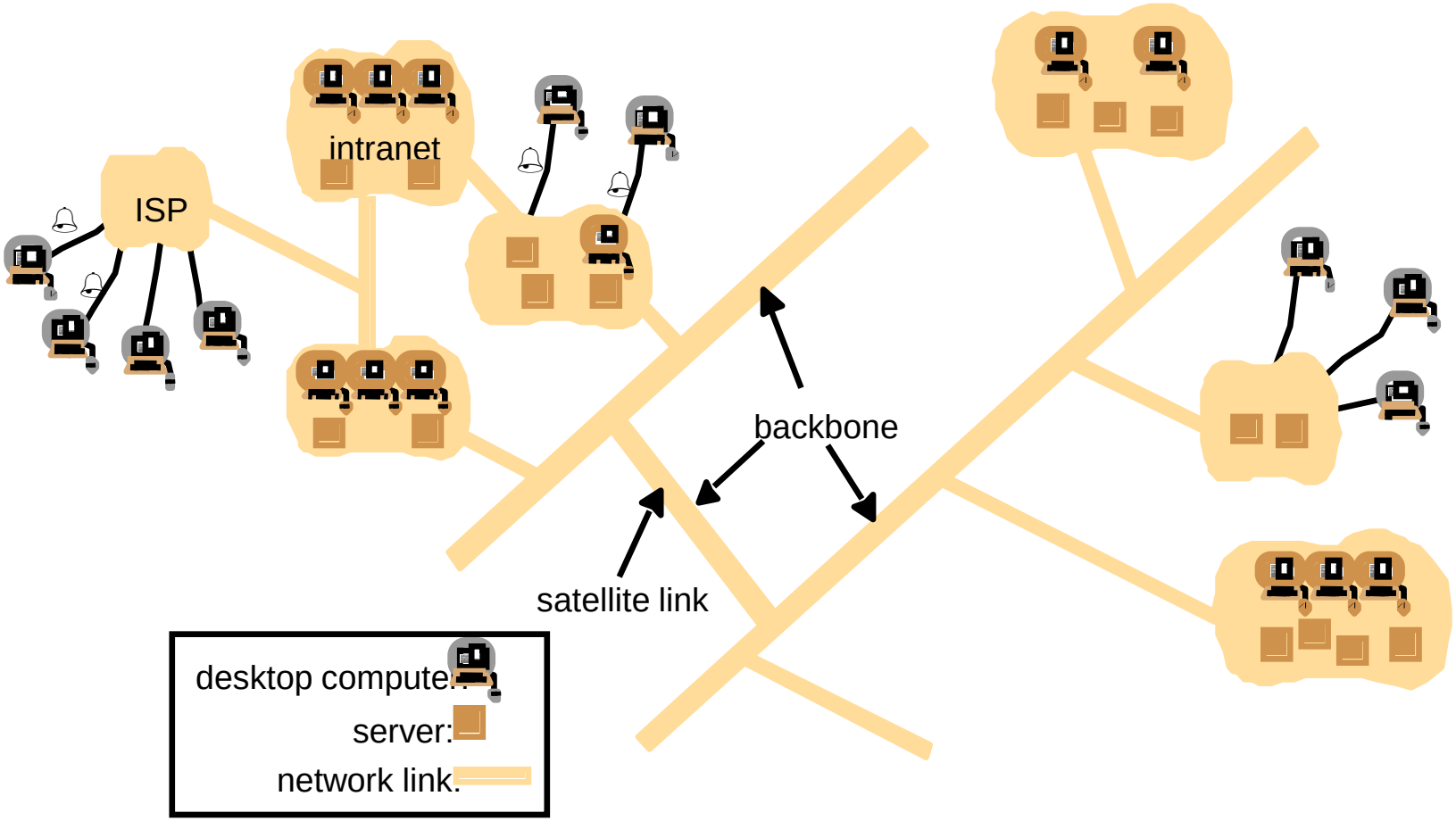
centralized computing

- Early computing was performed on a single processor. Uni-processor computing can be called centralized computing.

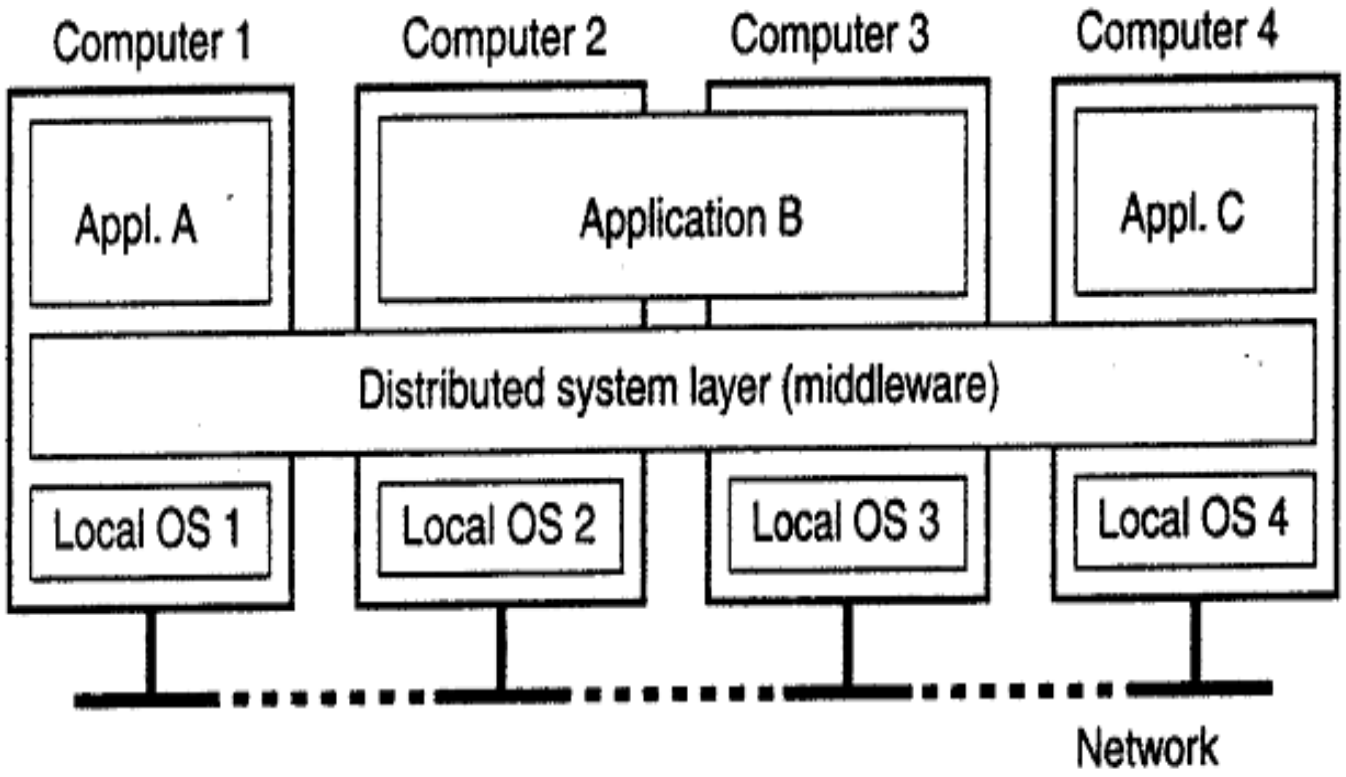
Distributed System

- A distributed system is a collection of independent computers that appears to its users as a single coherent system, interconnected via a network, capable of collaborating on a task.
- Distributed computing is computing performed in a distributed system.
- Distributed computing has become increasingly common due advances that have made both machines and networks cheaper and faster

A typical portion of the Internet



Distributed system organized as Middle wire



GOALS OF DISTRIBUTED SYSTEM

1. **Making Resources Accessible:**

   The main goal of a distributed system is to make it easy for the users (and applications) to access remote resources, and to share them in a controlled and efficient way.

2. **Distribution Transparency:**

   An important goal of a distributed system is to hide the fact that its processes and resources are physically distributed across multiple computers. A distributed system that is able to present itself to users and applications as if it were only a single computer system is said to be transparent.

GOALS OF DISTRIBUTED SYSTEM

2. The concept of transparency can be applied to several aspects of a distributed system, some example are access, location, migration, relocation, replication, concurrency and failure.

3. **Openness:**

An **open distributed** system is a system that offers services according to standard rules that describe the syntax and semantics of those services. In distributed systems, services are generally specified through interfaces.

GOALS OF DISTRIBUTED SYSTEM

4. <u>Scalability:</u>

- Scalability of a system can be measured along at least three different dimensions

a. First, a system can be scalable with respect to its size, meaning that we can easily add more users and resources to the system.

b. Second, a geographically scalable system is one in which the users and resources may lie far apart.

c. Third, a system can be administratively capable, meaning that it can still be easy to manage even if it spans many independent administrative organizations.

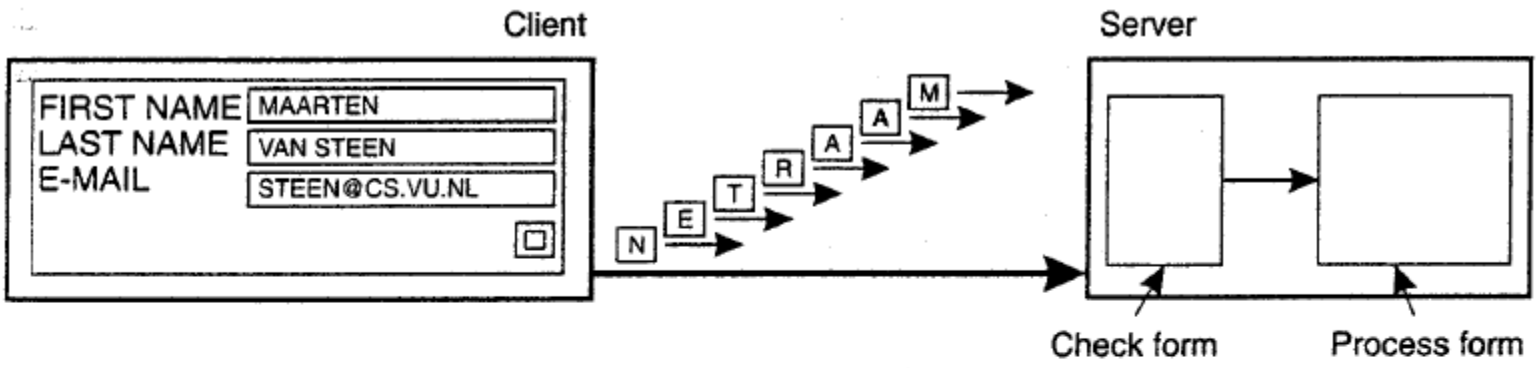
GOALS OF DISTRIBUTED SYSTEM

- There are now basically only three techniques for scaling:

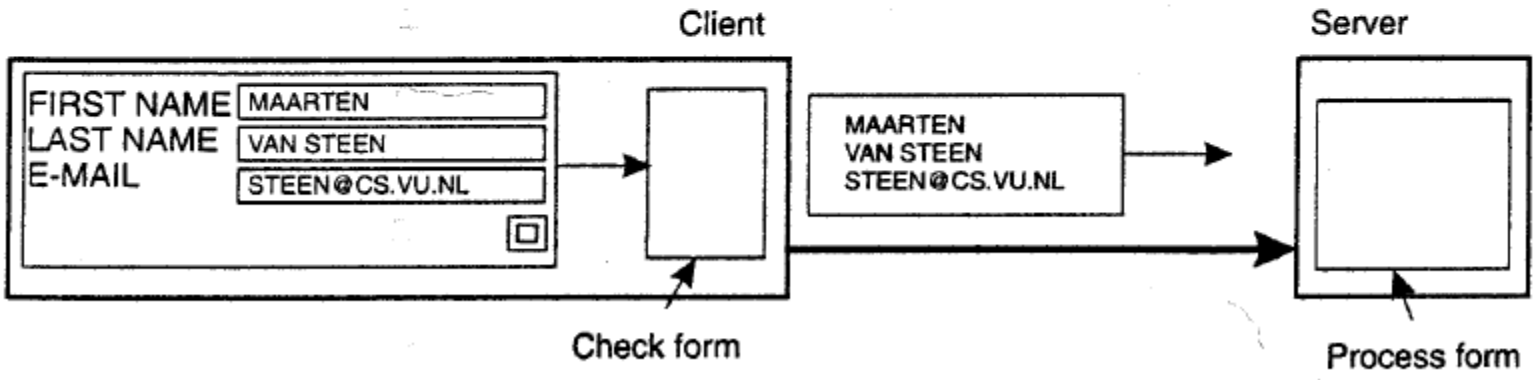
I. Hiding communication latencies:

- try to avoid waiting for responses to remote (and potentially distant) service requests as much as possible.
- In interactive applications solution is to reduce the overall communication, for example, by moving part of the computation that is normally done at the server to the client process requesting the service.

GOALS OF DISTRIBUTED SYSTEM



(a)



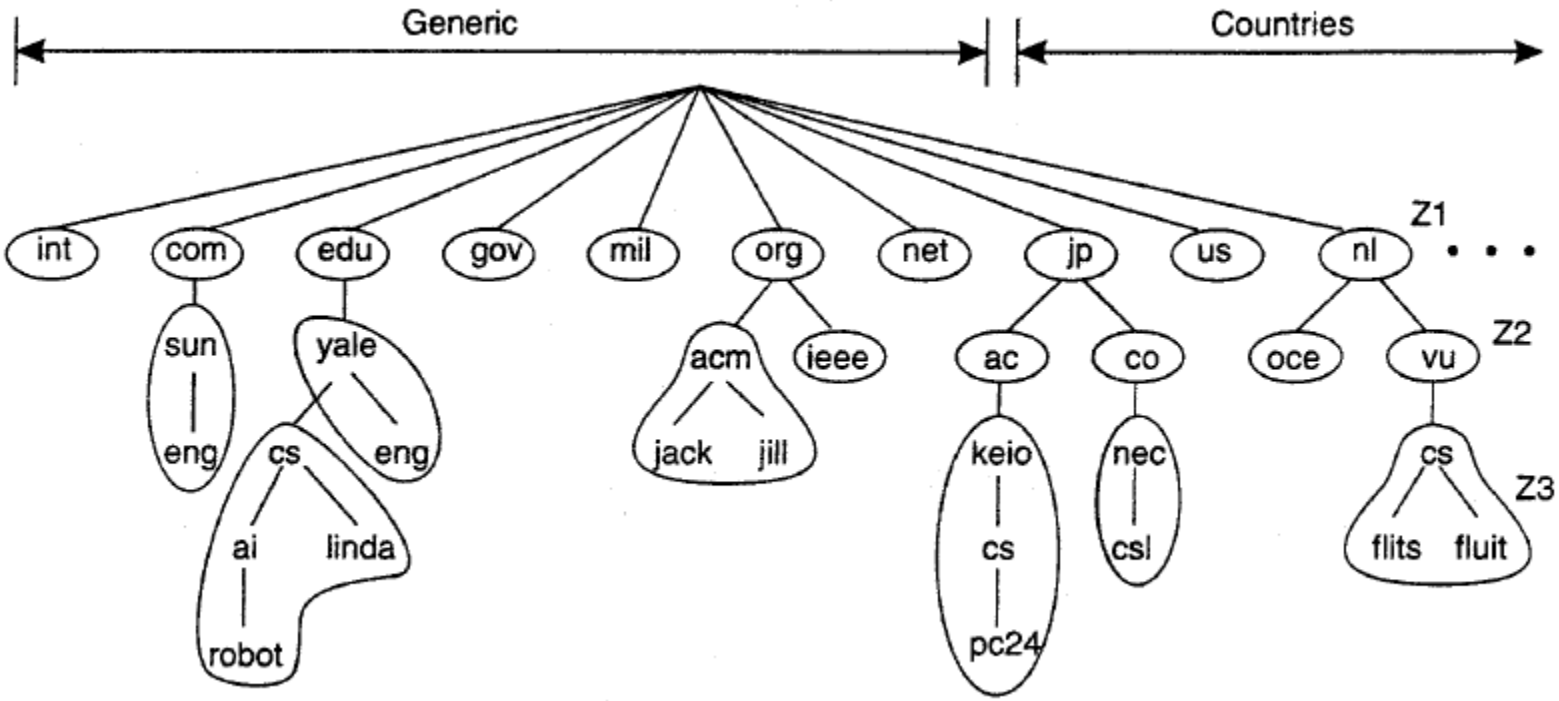
(b)

GOALS OF DISTRIBUTED SYSTEM

II. Distribution:

- Distribution involves taking a component, splitting it into smaller parts, and subsequently spreading those parts across the system. Example Domain Name System (DNS) name resolution.

GOALS OF DISTRIBUTED SYSTEM



An example of dividing the DNS name space into zones.

GOALS OF DISTRIBUTED SYSTEM

III. Replication :

- Replication not only increases availability, but also helps to balance the load between components leading to better performance. Also, in geographically widely-dispersed systems, having a copy nearby can hide much of the communication latency problems mentioned before.